CYBERSECURITY ASSESSMENT

Vulnerability Assessment Report

Passive Security Audit · Read-Only Scope · April 2026

Target Website	http://testphp.vulnweb.com
Assessment Type	Passive Vulnerability Assessment
Testing Scope	Public-facing pages only (no exploitation)
Methodology	OWASP Web Security Testing Guide (WSTG)
Report Date	April 2026
Prepared By	Suresh Basavaraj Hugar— Cybersecurity Task 1 (2026)

3	4	3	1	11
HIGH RISK	MEDIUM RISK	LOW RISK	INFORMATIONAL	TOTAL FINDINGS

Table of Contents

1. Executive Summary 3
2. Scope and Methodology 3
3. Tools Used 4
4. Summary of Findings 4
5. Detailed Findings 5–10
    5.1 Missing Content-Security-Policy Header 5
    5.2 Missing X-Frame-Options Header 5
    5.3 Server Software Version Disclosure 6
    5.4 Insecure Cookie Configuration 6
    5.5 FTP Port 21 Exposed to Internet 7
    5.6 No HTTPS / Missing HSTS Header 7
    5.7 Missing X-Content-Type-Options Header 8
    5.8 Referrer-Policy Not Configured 8
    5.9 Directory Listing Enabled 9
    5.10 Outdated Apache Web Server 9
    5.11 Missing Permissions-Policy Header 10
6. Remediation Priority Summary 10
7. Conclusion 11

1. Executive Summary

This report presents the findings of a passive vulnerability assessment conducted on **testphp.vulnweb.com**, a deliberately vulnerable web application provided by Acunetix for security training purposes. The assessment was performed using non-intrusive, read-only techniques — no login credentials were used, no data was modified, and no exploits were executed at any point during testing.

The assessment identified **11 security weaknesses** across four key areas: HTTP security headers, cookie configuration, network port exposure, and server information disclosure. Three findings are classified as High severity, four as Medium, three as Low, and one as Informational.

The most pressing concerns are the complete absence of HTTPS encryption, an open FTP port accessible from the internet, and session cookies that lack standard security flags. These three issues alone represent meaningful risk to any real user data and should be addressed before other findings.

3 High	4 Medium	3 Low	1 Info	11 Total

2. Scope and Methodology

Parameter	Detail
Target URL	http://testphp.vulnweb.com
Assessment Type	Passive Vulnerability Assessment (Read-Only)
In Scope	All publicly accessible pages and resources
Out of Scope	Authentication mechanisms, admin panels, database queries, active exploitation
Date Conducted	April 2026
Standard Applied	OWASP Web Security Testing Guide (WSTG v4.2)
Ethical Constraints	No brute force, no login bypass, no denial-of-service, no data modification

3. Tools Used

Tool	Category	Purpose
Nmap 7.x	Network Scanner	Port discovery and service/version fingerprinting
OWASP ZAP	Web App Scanner	Passive spider crawl and automated alert generation
Chrome DevTools (F12)	Browser Tool	Inspection of HTTP response headers and cookie flags
SecurityHeaders.com	Online Analyzer	Automated grading of HTTP security header configuration
SSL Labs (ssllabs.com)	Online Analyzer	TLS/SSL certificate and cipher suite evaluation
BuiltWith	Fingerprinting	Technology stack and framework identification

4. Summary of Findings

#	Vulnerability	Category	Risk Level
01	Missing Content-Security-Policy Header	HTTP Headers	MEDIUM
02	Missing X-Frame-Options Header	HTTP Headers	MEDIUM
03	Server Software Version Disclosure	HTTP Headers	LOW
04	Insecure Cookie Configuration	Cookie Config	HIGH
05	FTP Port 21 Exposed to Internet	Network / Ports	HIGH
06	No HTTPS / Missing HSTS Header	Transport Security	HIGH
07	Missing X-Content-Type-Options Header	HTTP Headers	LOW
08	Referrer-Policy Not Configured	HTTP Headers	LOW
09	Directory Listing Enabled	Server Config	MEDIUM
10	Outdated Apache Web Server	Server Software	MEDIUM
11	Missing Permissions-Policy Header	HTTP Headers	INFO

5. Detailed Findings

#01 Missing Content-Security-Policy (CSP) Header — MEDIUM

Location	HTTP Response Headers — All Pages
Description	The web server does not include a Content-Security-Policy header in any of its HTTP responses. CSP is a browser-level security mechanism that controls which external resources (scripts, stylesheets, images, fonts) are allowed to load on a given page.
Business Risk	The absence of a CSP policy means there is no browser-enforced barrier against Cross-Site Scripting (XSS) attacks. If an attacker manages to inject malicious JavaScript into any page — through a comment field, URL parameter, or third-party script — the browser will execute it without question. This can result in stolen session tokens, credential harvesting, or page defacement.
Recommended Fix	Define a Content-Security-Policy that whitelists trusted sources and add it to every HTTP response. A starting point: Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'. Test thoroughly to avoid breaking legitimate functionality before deploying to production.
Evidence	`Grade F on SecurityHeaders.com — header absent in all sampled HTTP responses`

#02 Missing X-Frame-Options Header — MEDIUM

Location	HTTP Response Headers — All Pages
Description	No X-Frame-Options header is present in the server's responses. This header controls whether the website can be embedded inside an iframe on a third-party web page.
Business Risk	Without this protection, the site is vulnerable to Clickjacking attacks. An attacker can embed the site in a transparent iframe overlaid on a fake page, tricking users into unknowingly clicking buttons — such as "confirm payment" or "allow access" — while believing they are interacting with a different site entirely.
Recommended Fix	Add the header X-Frame-Options: DENY to all responses to prevent framing entirely, or use SAMEORIGIN to allow framing only from the same domain. This is a one-line addition to the web server configuration and carries no performance overhead.
Evidence	`Confirmed absent via SecurityHeaders.com report and manual DevTools header inspection`

#03 Server Software Version Disclosure — LOW

Location	HTTP Response Header — "Server" Field
Description	The server's HTTP response includes a Server header that reveals the exact name and version of the web server software in use (e.g., Apache/2.4.4). This is visible to anyone who inspects the response headers — no special tools required.
Business Risk	While this does not represent a direct attack vector on its own, version disclosure significantly aids an attacker's reconnaissance phase. Knowing the precise software version allows them to search vulnerability databases (CVE, Exploit-DB) for known weaknesses specific to that version and craft targeted attacks. It is essentially handing attackers a list of applicable weapons.
Recommended Fix	In the Apache configuration file (httpd.conf), set ServerTokens Prod and ServerSignature Off. This causes Apache to respond with only the name "Apache" rather than the full version string, significantly reducing the information available to potential attackers.
Evidence	`Server: Apache/2.4.4 — visible in Chrome DevTools > Network > Response Headers`

#04 Insecure Cookie Configuration — Missing HttpOnly and Secure Flags — HIGH

Location	Session Cookie Configuration
Description	Session cookies set by the application are missing two critical security attributes: the HttpOnly flag and the Secure flag. HttpOnly prevents client-side JavaScript from reading the cookie. The Secure flag ensures the cookie is only transmitted over encrypted HTTPS connections, never over plain HTTP.
Business Risk	Missing HttpOnly means that any JavaScript running on the page — including injected malicious scripts — can read and exfiltrate session cookies, enabling full session hijacking. Missing Secure means the session cookie is transmitted in plain text over HTTP connections, where it can be intercepted by anyone on the same network (Man-in-the-Middle attack). On public Wi-Fi, this is trivial to exploit.
Recommended Fix	When creating or modifying session cookies, always include both flags in the Set-Cookie header: Set-Cookie: sessionid=value; HttpOnly; Secure; SameSite=Strict. The SameSite=Strict attribute should also be added to protect against CSRF attacks. Verify all cookies — not just the session cookie — are set correctly.
Evidence	`Verified in Chrome DevTools > Application > Cookies: HttpOnly and Secure columns unchecked`

#05 FTP Port 21 Open and Accessible — HIGH

Location	Network — TCP Port 21
Description	An Nmap port scan revealed that TCP port 21 (FTP — File Transfer Protocol) is open and listening for connections from the public internet. FTP is a legacy protocol developed in the 1970s and was not designed with security in mind.
Business Risk	FTP transmits all data — including usernames and passwords — in plain text, making it trivially interceptable by anyone in a position to observe network traffic. Open FTP services are routinely targeted by automated scanning tools and brute-force attack scripts. If anonymous FTP login is enabled (common misconfiguration), unauthenticated users may be able to read or upload files to the server directly.
Recommended Fix	If FTP is not actively required, close port 21 immediately at the firewall and disable the FTP service. If remote file transfer is genuinely needed, replace FTP with SFTP (SSH File Transfer Protocol, port 22), which provides strong encryption and key-based authentication. Verify no anonymous login is permitted in the meantime.
Evidence	`PORT 21/tcp open ftp — detected via: nmap -sV testphp.vulnweb.com`

#06 No HTTPS Encryption / Missing HSTS Header — HIGH

Location	Transport Layer — HTTP vs HTTPS
Description	The website is served entirely over plain HTTP with no HTTPS enforcement. The HTTP Strict Transport Security (HSTS) header is also absent. This means browsers will not automatically upgrade HTTP connections to HTTPS, and there is no encrypted channel between the user's browser and the web server.
Business Risk	All data exchanged between the user and the site — form inputs, login credentials, session tokens, personal information — travels across the network in clear text. This enables passive interception by anyone on the same network (e.g., public Wi-Fi, shared office networks) as well as active Man-in-the-Middle attacks where content can be read or modified in transit. Modern browsers will also display security warnings for non-HTTPS sites, damaging user trust.
Recommended Fix	Obtain and install a TLS certificate — Let's Encrypt provides free, auto-renewing certificates suitable for most sites. Configure the server to redirect all HTTP traffic to HTTPS (301 redirect). Once HTTPS is working correctly, add the HSTS header: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload.
Evidence	`Site loads over http:// with no redirect — confirmed via browser and SSL Labs scan`

#07 Missing X-Content-Type-Options Header — LOW

Location	HTTP Response Headers
Description	The X-Content-Type-Options header is not present in server responses. This header instructs browsers not to perform MIME type sniffing — the process by which a browser tries to guess the content type of a file based on its content rather than the declared Content-Type.
Business Risk	Without this header, a browser may incorrectly execute a file that was uploaded as an image or plain text but contains embedded JavaScript. An attacker who can upload any file to the server might exploit MIME sniffing to have the browser execute their file as a script. The fix is a single header value that entirely prevents this class of attack.
Recommended Fix	Add X-Content-Type-Options: nosniff to all HTTP responses via the server configuration. This is a zero-cost, one-line change with no negative impact on legitimate site functionality.
Evidence	`Header absent — confirmed in SecurityHeaders.com report (Grade F)`

#08 Referrer-Policy Header Not Set — LOW

Location	HTTP Response Headers
Description	No Referrer-Policy header is configured on the server. This header controls how much information about the current page's URL is included in the Referer header when a user navigates to another site by clicking a link.
Business Risk	Without a Referrer-Policy, the browser may send the full URL of the current page to external sites. If URLs contain sensitive parameters — user IDs, session tokens, search queries, internal path structure — this information is silently leaked to third-party services such as analytics platforms, advertisers, or any external resource that the page loads.
Recommended Fix	Add Referrer-Policy: strict-origin-when-cross-origin to all HTTP responses. This setting sends the full URL only to same-origin requests, and only the origin (domain, no path or parameters) to cross-origin HTTPS destinations.
Evidence	`Header absent — confirmed in SecurityHeaders.com and manual header inspection`

#09 Apache Directory Listing Enabled — MEDIUM

Location	Web Server Configuration
Description	Apache's directory listing feature is active on this server. When a URL is requested that points to a directory without an index file, the server returns an HTML page listing all files and subdirectories within that folder. This was confirmed by browsing directly to paths such as /images/ on the test site.
Business Risk	Exposed directory listings reveal the internal file structure of the web application to any visitor — no authentication required. This allows an attacker to locate backup files, old configuration files, scripts, or documents that were placed on the server but were never meant to be publicly accessible. In practice, this has led to real-world data breaches where developers left database dumps or credential files in web-accessible directories.
Recommended Fix	Disable directory listing globally in the Apache configuration by changing Options Indexes to Options -Indexes in either httpd.conf or the site's .htaccess file. Audit the server for any sensitive files currently exposed in browsable directories.
Evidence	`Directory listing visible at http://testphp.vulnweb.com/images/ — confirmed via browser`

#10 Outdated Apache Web Server Software — MEDIUM

Location	Server Software — Apache HTTP Server
Description	The Server header identifies the running software as Apache version 2.4.4. This version is significantly outdated — current Apache 2.4.x releases are in the 2.4.58+ range. Each major version gap represents a collection of security patches that have not been applied to this server.
Business Risk	Running outdated server software means that publicly documented vulnerabilities (CVEs) that have already been patched in newer versions remain exploitable on this server. Automated scanning tools routinely identify version numbers and match them against databases of known exploits. This is one of the most common root causes of real-world server compromises.
Recommended Fix	Update Apache to the latest stable release in the 2.4.x branch as soon as possible. On Debian/Ubuntu systems: sudo apt update && sudo apt upgrade apache2. Establish a regular patch management cycle — at minimum, apply security updates monthly and subscribe to Apache security advisories at httpd.apache.org.
Evidence	`Server: Apache/2.4.4 — disclosed in HTTP response header; current release is 2.4.58+`

#11 Missing Permissions-Policy Header — INFO

Location	HTTP Response Headers
Description	The Permissions-Policy header (formerly known as Feature-Policy) is not configured. This header allows site owners to explicitly control which browser features and APIs — such as camera access, microphone, geolocation, and payment APIs — are available to the page and any embedded third-party content.
Business Risk	While the severity of this finding is low in isolation, the absence of a Permissions-Policy means the site relies entirely on default browser behavior for feature access. Third-party scripts loaded by the page (analytics, advertising, widgets) could potentially access device capabilities unless explicitly restricted. The risk increases in proportion to the amount of third-party JavaScript on the page.
Recommended Fix	Add a Permissions-Policy header that explicitly disables features not required by the application. A safe default: Permissions-Policy: geolocation=(), microphone=(), camera=(), payment=(). Adjust the policy based on which features the site legitimately uses.
Evidence	`Header absent — confirmed via SecurityHeaders.com report`

6. Remediation Priority Summary

The table below lists all findings ordered by severity and recommended fix priority. High-severity items should be treated as urgent. Many Medium and Low findings involve simple one-line server configuration changes that can be implemented in minutes.

Priority	Finding	Risk	Effort	Action Required
1	Enable HTTPS and Add HSTS	HIGH	Medium	Install SSL cert; redirect HTTP→HTTPS; add HSTS header
2	Secure Session Cookies	HIGH	Low	Set HttpOnly; Secure; SameSite=Strict on all cookies
3	Close FTP Port 21	HIGH	Low	Block port 21 in firewall; migrate to SFTP if needed
4	Add Content-Security-Policy	MEDIU M	Medium	Define and implement CSP header in server config
5	Add X-Frame-Options	MEDIU M	Low	Add X-Frame-Options: DENY header to all responses
6	Disable Directory Listing	MEDIU M	Low	Set Options -Indexes in Apache config or .htaccess
7	Update Apache Server	MEDIU M	Medium	Run apt update && apt upgrade apache2
8	Suppress Server Version	LOW	Low	Set ServerTokens Prod in httpd.conf
9	Add X-Content-Type-Options	LOW	Low	Add X-Content-Type-Options: nosniff header
10	Add Referrer-Policy	LOW	Low	Add Referrer-Policy: strict-origin-when-cross-origin
11	Add Permissions-Policy	INFO	Low	Add Permissions-Policy header with disabled features

7. Conclusion

This passive vulnerability assessment of **testphp.vulnweb.com** identified 11 security issues that collectively represent a weak security posture. While the target is an intentionally vulnerable training site, the findings mirror vulnerabilities that are routinely discovered on real production websites — particularly those that have not received regular security attention.

The good news is that the majority of findings in this report are straightforward to remediate. Most of the missing HTTP headers can be added in minutes through server configuration changes, requiring no application code changes. The three high-severity issues — HTTPS enforcement, cookie security flags, and the open FTP port — should be prioritised above all others.

Key recommendations for the site owner:

- Enable HTTPS immediately. A free TLS certificate from Let's Encrypt takes under 30 minutes to configure and eliminates the risk of data interception in transit.
- Audit all cookies and add HttpOnly, Secure, and SameSite flags. This is a single change in the application layer that eliminates session hijacking via script injection.
- Close or firewall port 21. FTP has no place on a modern internet-facing server. If file transfer is needed, SFTP is the correct replacement.
- Apply all missing HTTP security headers through a single server-level configuration block. This addresses five findings at once with minimal effort.
- Keep the server software updated. Subscribe to Apache security advisories and apply patches as they are released.
- Establish a regular review cycle. Security is not a one-time task. Quarterly assessments — even informal ones using these same free tools — catch new issues before they become problems.